Exercise 10: Computer Vision — Object Detection with YOLO

Course Outline: `course-outline-v4.pdf` — Chapter 6: Deep Learning (Computer Vision)
Corresponding Week: Week 5
Estimated Duration: 4-5 hours

🔬 How to Research (Important — Read This First)

This lab runs BEFORE your lecture. You are NOT expected to know the answers. You are expected to **find** them.

Your Research Toolbox (in order of preference)

Step	Method	When to Use
1	**Try local resources first**	If the task names a specific file, open it and look.
2	**Ask an AI assistant**	If local resources don't have the answer, ask ChatGPT, Claude, or any AI tool.
3	**Search the web**	If AI gives you an answer, verify it with a web search or textbook.

⚠️ Citation Rules (Mandatory for ALL Deliverables)

Every answer in your deliverable MUST include a source:

Answer Type	Source Format
From a local file	`C4 - CNN notes, Week 3`
From AI-assisted research	Use AI as a research tool, NOT as a source. Ask AI: `"Give me a specific URL, video, or textbook for [topic]."` Verify the link exists. Cite the URL/video/textbook, NOT `"AI said so."`
From a web page	`https://arxiv.org/abs/1506.02640`
From official documentation	`https://docs.ultralytics.com/`

No source = No credit.

🤖 Example AI Prompts You Can Use

- "What is the difference between image classification, object detection, and instance segmentation?"
- "Explain YOLO architecture: how does it predict bounding boxes in one forward pass?"
- "What is IoU (Intersection over Union) and how is it used in object detection?"
- "What are anchor boxes in object detection?"
- "How does Non-Maximum Suppression (NMS) work?"

⚠️ Code Integrity Note

You must **write the code yourself**. You may ask AI to explain concepts, but do NOT copy-paste solutions.

🎯 Learning Objectives

1. Understand the difference between classification, detection, and segmentation
2. Learn object detection evaluation metrics (IoU, mAP, Precision, Recall)
3. Set up and run a pre-trained YOLO model for inference
4. Perform object detection on custom images and videos
5. Understand the trade-offs between speed and accuracy in detection models

---

📦 Prerequisites — Setup (15 min)

Required Dependencies

Before starting, install the required libraries:

```
pip install ultralytics opencv-python pillow matplotlib numpy pandas seaborn
```

Source: https://docs.ultralytics.com/quickstart/

Verify Installation

```
import ultralytics
import cv2
import matplotlib.pyplot as plt

print(f"Ultralytics version: {ultralytics.__version__}")
print(f"OpenCV version: {cv2.__version__}")
```

🧠 Module A: Understanding Object Detection (45 min)

📁 Resources

Resource	Path / URL	Purpose
YOLO Research Paper	https://arxiv.org/abs/1506.02640	Original YOLO paper
YOLO Architecture Guide	https://docs.ultralytics.com/	Official documentation
Object Detection Tutorial	C4 - CNN, Week 3	CNN fundamentals

Task A.1 — Classification vs Detection vs Segmentation (15 min)

Ask ChatGPT: "Explain the difference between image classification, object detection, and instance segmentation with visual examples."
Research: What are the outputs of each task? (e.g., classification → class label only)

Fill in:

Task	Output	Example	Real-world Application	Source
Classification	_	"This is a cat"	_	_
Object Detection	_	Bounding box around cat	_	_
Instance Segmentation	_	Pixel mask of cat	_	_

Task A.2 — Object Detection Fundamentals (15 min)

Ask ChatGPT: "What is Intersection over Union (IoU)? Draw a diagram and explain."
Ask ChatGPT: "What is Non-Maximum Suppression (NMS)? Why do we need it in object detection?"

Fill in:

Concept	Definition	Formula (if applicable)	Why It's Needed	Source
IoU	_	_	_	_
Precision	_	TP / (TP + FP)	_	_
Recall	_	_	_	_
mAP (mean Average Precision)	_	—	_	_
NMS	_	—	_	_

Task A.3 — YOLO Architecture (15 min)

Ask ChatGPT: "Explain the YOLO architecture. What makes it different from two-stage detectors like Faster R-CNN?"
Ask ChatGPT: "What are anchor boxes in YOLO? How do they help with bounding box prediction?"

Fill in:

Feature	YOLO (One-Stage)	Faster R-CNN (Two-Stage)	Source
Number of stages	_	_	_
Speed	_	_	_
Accuracy	_	_	_
Typical use case	_	_	_
Key innovation	_	_	_

📸 Module B: Running YOLO Inference on Images (60 min)

📁 Resources

Resource	Path / URL	Purpose
Ultralytics Quickstart	https://docs.ultralytics.com/quickstart/	Running YOLO models
COCO Dataset Classes	https://cocodataset.org/#explore	Class labels for pre-trained models

Task B.1 — Load a Pre-trained YOLO Model (15 min)

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

# Load a pre-trained YOLOv8n model (n = nano version, smallest/fastest)
model = YOLO('yolov8n.pt')

# Print model information
print(f"Model loaded successfully!")
print(f"Number of classes: {len(model.names)}")
print(f"Class names: {model.names[:10]}...")  # First 10 classes
```

Research: Ask ChatGPT: "What do the different YOLO variants mean? (yolov8n, yolov8s, yolov8m, yolov8l, yolov8x)"

Fill in:

Model Variant	Size	Speed	Accuracy	Best For	Source
YOLOv8n (nano)	–	–	–	–	–
YOLOv8s (small)	–	–	–	–	–
YOLOv8m (medium)	–	–	–	–	–
YOLOv8l (large)	–	–	–	–	–
YOLOv8x (extra large)	–	–	–	–	–

Task B.2 — Run Inference on Sample Images (20 min)

```
# Method 1: Use a sample image from the web or your local folder
# Download a sample image (uncomment if needed)
# !wget [https://ultralytics.com/images/bus.jpg](https://ultralytics.com/images/bus.jpg) -O bus.jpg

# Run inference
results = model('[https://ultralytics.com/images/bus.jpg](https://ultralytics.com/images/bus.jpg)')  # URL or local
path

# Display results
results[0].show()  # OpenCV window

# Get results as arrays
boxes = results[0].boxes
print(f"Detected {len(boxes)} objects")

# Show detection details
for box in boxes:
    x1, y1, x2, y2 = box.xyxy[0].tolist()  # Bounding box coordinates
    conf = box.conf[0].item()  # Confidence score
    cls = int(box.cls[0].item())  # Class ID
    class_name = model.names[cls]
    print(f"Class: {class_name}, Confidence: {conf:.2f}, Box: ({x1:.0f}, {y1:.0f}, {x2:.0f}, {y2:.0f})")
```

Record: Run the code on the sample image. Fill in the table:

Object Detected	Confidence Score	Bounding Box Coordinates	Source
–	–	–	Code output
–	–	–	Code output
–	–	–	Code output

Task B.3 — Visualize Detection Results (15 min)

```
# Method 2: Plot with matplotlib
import matplotlib.pyplot as plt
from PIL import Image
```

```
import numpy as np

# Get the annotated image
annotated_img = results[0].plot()  # Returns numpy array with boxes drawn

# Convert BGR (OpenCV) to RGB (Matplotlib)
annotated_img_rgb = cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB)

# Display
plt.figure(figsize=(10, 8))
plt.imshow(annotated_img_rgb)
plt.axis('off')
plt.title("YOLO Object Detection Results")
plt.show()

# Save results
cv2.imwrite('detection_result.jpg', annotated_img)
print("Saved as detection_result.jpg")
```

Task B.4 — Test on Multiple Images (10 min)

```
# Test on multiple images
image_urls = [
    '[https://ultralytics.com/images/bus.jpg](https://ultralytics.com/images/bus.jpg)',
    '[https://ultralytics.com/images/zidane.jpg](https://ultralytics.com/images/zidane.jpg)',
    '[https://ultralytics.com/images/coco.jpg](https://ultralytics.com/images/coco.jpg)'
]

for url in image_urls:
    results = model(url)
    img = results[0].plot()
    print(f"{url.split('/')[-1]}: {len(results[0].boxes)} objects detected")
```

🎥 Module C: YOLO on Video and Webcam (45 min)

Task C.1 — Object Detection on Video (25 min)

```
import cv2
from ultralytics import YOLO

# Load model
model = YOLO('yolov8n.pt')

# Open video file (or use 0 for webcam)
video_path = 'sample_video.mp4'  # You can download a sample video
cap = cv2.VideoCapture(video_path)  # or cv2.VideoCapture(0) for webcam

# Get video properties
fps = int(cap.get(cv2.CAP_PROP_FPS))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

# Define output video writer
out = cv2.VideoWriter('output_video.mp4',
                      cv2.VideoWriter_fourcc(*'mp4v'),
                      fps, (width, height))
frame_count = 0

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Run YOLO on frame
    results = model(frame, verbose=False)

    # Annotate frame
    annotated_frame = results[0].plot()
```

```
    # Write frame
    out.write(annotated_frame)

    # Display (optional)
    if frame_count % 30 == 0:  # Show every 30th frame
        cv2.imshow('YOLO Detection', annotated_frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    frame_count += 1

cap.release()
out.release()
cv2.destroyAllWindows()
print(f"Processed {frame_count} frames. Output saved to 'output_video.mp4'")
```

Research: Ask ChatGPT: "What are the challenges of running object detection on video compared to single images?"

Task C.2 — Real-time FPS Analysis (20 min)

```
import time
import pandas as pd

# Measure inference speed
model = YOLO('yolov8n.pt')
times = []
n_runs = 100

for i in range(n_runs):
    start = time.time()
    results = model('[https://ultralytics.com/images/bus.jpg](https://ultralytics.com/images/bus.jpg)')
    end = time.time()
    times.append(end - start)

avg_time = sum(times) / n_runs
fps = 1 / avg_time
print(f"Average inference time: {avg_time*1000:.2f} ms")
print(f"Frames per second (FPS): {fps:.1f}")

# Compare models
variants = ['yolov8n.pt', 'yolov8s.pt', 'yolov8m.pt']
comparison = []

for variant in variants:
    try:
        model = YOLO(variant)
        start = time.time()
        results = model('[https://ultralytics.com/images/bus.jpg](https://ultralytics.com/images/bus.jpg)')
        end = time.time()
        comparison.append({
            'Model': variant,
            'Inference Time (ms)': (end - start) * 1000,
            'FPS': 1 / (end - start)
        })
    except:
        print(f"Model {variant} not found. Skipping.")

df = pd.DataFrame(comparison)
print(df)
```

Fill in:

Model Variant	Inference Time (ms)	FPS	Source
YOLOv8n	_	_	Code output
YOLOv8s	_	_	Code output
YOLOv8m	_	_	Code output

🛠️ Module D: Custom Object Detection (45 min)

Task D.1 — Understanding Custom Training (20 min)

Ask ChatGPT: "What is transfer learning in object detection? How do you fine-tune YOLO on a custom dataset?"
Ask ChatGPT: "What is the COCO dataset? How many classes does it have?"

Fill in:

Concept	Explanation	Source
Transfer Learning	_	_
Fine-tuning	_	_
Dataset Format (YOLO)	_	_
COCO Dataset	_	_

Task D.2 — Dataset Structure (15 min)

```
# YOLO dataset structure (for custom training)
#
# dataset/
# ├── images/
# │   ├── train/
# │   │   ├── image1.jpg
# │   │   └── image2.jpg
# │   └── val/
# │       └── image3.jpg
# └── labels/
#     ├── train/
#     │   ├── image1.txt
#     │   └── image2.txt
#     └── val/
#         └── image3.txt
#
# Each label file format:
# class_id x_center y_center width height
# (all normalized to [0, 1])
```

Research: Ask ChatGPT: "How are YOLO annotation files formatted? What does each number represent?"

Fill in:

Field	Description	Range	Example	Source
class_id	_	_	0	_
x_center	_	_	0.5	_
y_center	_	_	0.5	_
width	_	_	0.3	_
height	_	_	0.4	_

Task D.3 — Custom Training Preview (10 min)

```
# This is a preview of custom training (you may not run this without a dataset)
# from ultralytics import YOLO
#
# # Load a pre-trained model
# model = YOLO('yolov8n.pt')
#
# # Train the model
# results = model.train(
#     data='custom_dataset.yaml',  # Path to dataset configuration
#     epochs=50,                   # Number of training epochs
#     imgsz=640,                   # Image size
#     batch=16,                    # Batch size
#     device='cpu'                 # 'cpu' or 'cuda' for GPU
# )
```

Research: Ask ChatGPT: "What are the minimum requirements to train YOLO on a custom dataset? How many images do you need?"

📊 Module E: Evaluation and Analysis (30 min)

Task E.1 — Understanding Detection Metrics (15 min)

```
# After running detection, evaluate results
# This requires ground truth annotations (for validation)
# Simulated evaluation

def calculate_iou(box1, box2):
    """Calculate IoU between two bounding boxes"""
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    intersection = max(0, x2 - x1) * max(0, y2 - y1)
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
    union = area1 + area2 - intersection

    return intersection / union if union > 0 else 0

# Example: two boxes
box_gt = [10, 10, 50, 50]      # Ground truth
box_pred = [15, 15, 55, 55]    # Prediction
iou = calculate_iou(box_gt, box_pred)
print(f"IoU: {iou:.2f}")

# IoU > 0.5 is typically considered a correct detection (True Positive)
```

Research: Ask ChatGPT: "What IoU threshold is commonly used to determine True Positives in object detection?"

Task E.2 — Analysis of Detection Results (15 min)

```
from PIL import Image
import matplotlib.pyplot as plt

# Analyze detection confidence distribution
results = model('[https://ultralytics.com/images/bus.jpg](https://ultralytics.com/images/bus.jpg)')
boxes = results[0].boxes
confidences = [box.conf[0].item() for box in boxes]
classes = [model.names[int(box.cls[0].item())] for box in boxes]

# Plot confidence distribution
plt.figure(figsize=(10, 5))
plt.bar(classes, confidences)
plt.ylim(0, 1)
plt.xlabel('Detected Objects')
plt.ylabel('Confidence Score')
plt.title('Detection Confidence by Class')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Create summary table
import pandas as pd
summary_df = pd.DataFrame({
    'Class': classes,
    'Confidence': confidences
})
print(summary_df)
```

Fill in:

Question	Answer	Source
Which object was detected with highest confidence?	_	Code output
Which object was detected with lowest confidence?	_	Code output
What was the average confidence score?	_	Code output

📤 Deliverables

#	Deliverable	From Module	Format Required
1	Classification vs Detection vs Segmentation Table (A.1)	Module A	Table with sources
2	IoU, Precision, Recall, mAP, NMS Table (A.2)	Module A	Table with sources
3	YOLO vs Faster R-CNN Comparison (A.3)	Module A	Table with sources
4	YOLO Variants Comparison (B.1)	Module B	Table with sources
5	Detection Results on Sample Image (B.2)	Module B	Screenshot + Table
6	Video Processing Results (C.1)	Module C	Screenshot or summary
7	Speed Comparison Table (C.2)	Module C	Table
8	YOLO Dataset Format Table (D.2)	Module D	Table with sources
9	Confidence Analysis Table (E.2)	Module E	Table
10	**Brief Report (≤2 pages):** - Summarize key findings - Compare YOLOv8 variants speed vs accuracy - Discuss challenges in object detection - Include at least one detection visualization	All modules	PDF or Markdown

---

✅ Self-Check

- [ ] I can explain the difference between classification, detection, and segmentation
- [ ] I understand IoU, Precision, Recall, and mAP
- [ ] I can load and run a pre-trained YOLO model
- [ ] I can perform object detection on images and videos
- [ ] I can analyze detection results and confidence scores
- [ ] I understand YOLO dataset format for custom training
- [ ] I know the trade-offs between speed and accuracy in detection models

---

🎲 Bonus Tasks (Optional)

Bonus 1 — Visualize Feature Maps

```
# Visualize intermediate layers of YOLO backbone
# Requires model inspection and layer extraction
```

Bonus 2 — Track Objects Across Frames

```
# Use YOLO tracking mode
results = model.track('video.mp4', persist=True)
```

Bonus 3 — Custom Training with Roboflow

```
# Use Roboflow to create a custom dataset
# Train YOLO on custom objects (e.g., your own photos)
```

Bonus 4 — Detection + OCR

```
# Combine YOLO with Tesseract OCR for text detection
# Detect license plates, then read text
```

Bonus 5 — Model Quantization for Edge Deployment

```
# Export to TensorRT, ONNX, or TFLite
model.export(format='onnx')  # Export to ONNX
```

📚 Additional Resources

Resource	URL	Purpose
YOLO Paper (Original)	https://arxiv.org/abs/1506.02640	Understand the original YOLO
YOLOv8 Documentation	https://docs.ultralytics.com/	Official docs and tutorials
COCO Dataset	https://cocodataset.org/	Standard benchmark dataset
Roboflow	https://roboflow.com/	Tools for custom dataset creation
OpenCV Tutorials	https://docs.opencv.org/	Image processing fundamentals